

MANUAL TÉCNICO

SYSTEMACT

Sistema de gestión de inventario de activos físicos

Conviventia

Versión	1.0
Fecha	Mayo de 2026
Autor	Diego Alejandro Henao Henao
Stack	Next.js 15 · Supabase · TypeScript
Despliegue	Vercel + Supabase Cloud

Tabla de contenido

Esta tabla se actualiza al abrir el documento en Word (clic derecho → Actualizar campos).

Tabla de contenido	1
1. Introducción	1
1.1. Alcance	1
1.2. Audiencia	1
2. Stack tecnológico	1
2.1. Justificación	1
3. Arquitectura general	1
3.1. Capas de seguridad	1
3.2. Flujo típico de una mutación	1
4. Estructura del repositorio	1
5. Instalación desde cero	1
5.1. Pre-requisitos	1
5.2. Clonar el repositorio	1
5.3. Instalar dependencias	1
5.4. Variables de entorno	1
5.5. Crear el esquema de la base de datos	1
5.6. Levantar el frontend	1
5.7. Crear el primer administrador	1
6. Base de datos	1
6.1. Modelo de datos	1
6.2. Reglas clave del modelo	1
6.3. Funciones y RPCs principales	1
6.4. Roles y permisos	1
6.5. Seed inicial	1
6.6. Almacenamiento (Storage)	1
7. Módulos funcionales	1
7.1. Reglas de negocio destacadas	1
8. Autenticación y autorización	1
8.1. Flujo de auth	1
8.2. Guard de página (server component)	1
8.3. UI condicional por rol	1
9. API y endpoints	1
10. Despliegue	1
10.1. Frontend en Vercel	1

10.2. Backend en Supabase Cloud	1
10.3. Migraciones futuras	1
11. Pruebas	1
12. Convenciones de código	1
12.1. Reglas duras	1
13. Mantenimiento	1
13.1. Tareas periódicas	1
13.2. Puntos de extensión.....	1
13.3. Recuperación ante desastres	1
14. Referencias internas	1

1. Introducción

SYSTEMACT es una aplicación web interna que permite a Conviventia registrar, transferir, dar de baja y auditar los activos físicos de la organización. Reemplaza un sistema legacy en PHP/MySQL por un stack moderno basado en Next.js y Supabase.

1.1. Alcance

Este documento cubre la información técnica necesaria para instalar, configurar, mantener y extender el sistema. No es un manual de uso final — para eso ver el Manual de Usuario.

1.2. Audiencia

Personal técnico de Conviventia y futuros desarrolladores que reciban el sistema.

2. Stack tecnológico

Capa	Tecnología	Versión
Framework	Next.js (App Router)	15+
Lenguaje	TypeScript (strict)	5+
Runtime	Node.js	20+
UI	React	19
Estilos	Tailwind CSS + shadcn/ui	3.4 / latest
Iconos	lucide-react	—
Formularios	React Hook Form + Zod	7 / 4
Tablas	TanStack Table	8
Gráficos	Recharts	3
Notificaciones	Sonner	2
Exportación	ExcelJS	4
Backend	Supabase (BaaS)	latest
Base de datos	PostgreSQL	15
Autenticación	Supabase Auth (JWT en cookies)	—
Almacenamiento	Supabase Storage	—

Capa	Tecnología	Versión
Pruebas	Vitest + Playwright	4 / 1
Hosting	Vercel + Supabase Cloud	—

2.1. Justificación

- Next.js 15 con Server Components y Server Actions elimina la necesidad de mantener una API REST separada para mutaciones.
- Supabase unifica Postgres, Auth y Storage; la lógica crítica vive en RPCs PL/pgSQL.
- TypeScript estricto + Zod garantiza tipado en cliente, servidor y validación de entrada.

3. Arquitectura general

El sistema se compone de un frontend Next.js desplegado en Vercel y un backend Supabase (PostgreSQL + Auth + Storage). El cliente se comunica con la BD a través del SDK oficial de Supabase, y todas las mutaciones críticas pasan por Server Actions que invocan RPCs PL/pgSQL.

3.1. Capas de seguridad

1. RLS (Row-Level Security) en cada tabla — frontera de la base de datos.
2. RPCs security invoker que invocan require_rol_escritura() o require_rol_admin() antes de mutar datos.
3. Guards de página (requireRol) y validaciones en Server Actions — defensa en profundidad.

3.2. Flujo típico de una mutación

```
Usuario → Form (cliente, valida con Zod)
      → Server Action (re-valida con Zod, obtiene user)
      → RPC PL/pgSQL (require_rol_*, lock, insert + log)
      → revalidatePath()
      → toast Sonner
```

4. Estructura del repositorio

```
systemact/
├── app/
│   ├── (dashboard)/      Rutas protegidas
│   │   ├── inicio/      Panel KPIs y timeline
│   │   ├── bienes/       CRUD de bienes
│   │   ├── sedes/ areas/ categorias/
│   │   ├── transferencias/
│   │   ├── bajas/        (admin only)
│   │   └── historial/     Timeline por bien
```

├── reportes/	Inventario por persona
├── usuarios/	(admin only)
├── layout.tsx	
├── api/export/	Route Handlers .xlsx
├── auth/	Login, sign-up, recuperación
├── page.tsx	Portada pública
├── components/	ui/, layout/, *-form.tsx
├── lib/	
├── auth/require-rol.ts	Guards de rol
├── export/excel.ts	Helpers ExcelJS
├── supabase/	Clients (server, client, proxy)
├── validations/	Esquemas Zod
├── constants.ts	ROLES, ESTADOS_BIEN, NAV_GROUPS
├── utils.ts	
├── supabase/	
├── migrations/	1 baseline + _archive/
├── seed.sql	
├── config.toml	
├── tests/	Playwright E2E
├── proxy.ts	Middleware Next.js
├── documentation/	
├── CODE_GUIDE.md	
├── CHANGELOG.md	
├── MANUAL_TECNICO.md	

5. Instalación desde cero

5.1. Pre-requisitos

- Node.js ≥ 20 y npm ≥ 10
- Git
- Supabase CLI (npm install -g supabase o scoop install supabase)
- Docker Desktop (solo para Supabase local)
- Cuenta en supabase.com (para Supabase Cloud)

5.2. Clonar el repositorio

```
git clone https://github.com/<organizacion>/systemact.git
cd systemact
```

5.3. Instalar dependencias

```
npm install
```

5.4. Variables de entorno

Copiar la plantilla y completarla con las credenciales de Supabase:

```
cp .env.example .env.local
```

Contenido de .env.local:

```
NEXT_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=<publishable-key>

# Opcional para pruebas E2E autenticadas
E2E_USER_EMAIL=usuario@dominio.com
E2E_USER_PASSWORD=contraseña
```

Las credenciales se obtienen en Supabase Dashboard → Settings → API. Nunca versionar `.env.local`: ya está en `.gitignore`.

5.5. Crear el esquema de la base de datos

Opción A — proyecto local con Docker (desarrollo):

```
supabase start          # Postgres, Auth, Studio en local
supabase db reset       # aplica migración + seed
```

Studio disponible en `http://127.0.0.1:54323`

Opción B — proyecto en Supabase Cloud (producción):

```
supabase login
supabase link --project-ref <project-ref>
supabase db push          # aplica migración baseline
psql "$DATABASE_URL" -f supabase/seed.sql
```

Esto crea 9 tablas, 14 funciones/RPCs, 25 políticas RLS, 1 bucket de Storage con 4 policies y 3 triggers.

5.6. Levantar el frontend

```
npm run dev
```

App disponible en `http://localhost:3000`

5.7. Crear el primer administrador

Por defecto, todo nuevo usuario se registra con rol CONSULTOR (solo lectura). Para promover el primer admin:

4. Registrarse desde la app en `/auth/sign-up`.
5. En Supabase Studio (SQL Editor) o vía `psql` ejecutar:

```
update public.profiles
set rol = 'ADMINISTRADOR'
where id = (select id from auth.users
            where email = 'tu-correo@dominio.com');
```

6. Cerrar sesión y volver a entrar. El módulo `/usuarios` ya estará visible.

6. Base de datos

6.1. Modelo de datos

Tabla	Función
profiles	Extiende auth.users con datos de la persona y rol.
sedes	Catálogo de sedes físicas.
areas	Catálogo de áreas organizacionales.
caracteristicas	Tipos de bien con prefijo (genera el código automático).
bienes	Entidad principal — los activos físicos.
transferencias	Snapshot de cada cambio de ubicación.
bajas	Registro irreversible de salidas de inventario.
movimiento_bienes	Log de auditoría por bien.
keepalive	Tabla utilitaria contra hibernación de Supabase.

6.2. Reglas clave del modelo

- Código de bien generado por generar_codigo_bien(prefijo) con formato PREFIJO-AÑO-NNN (ej. COMP-2026-001).
- bienes.valor_total es columna GENERATED (cantidad × valor_unitario); nunca se envía desde el cliente.
- bienes.estado ∈ { ACTIVO, INACTIVO, DE BAJA }. Un bien nunca se borra físicamente; las bajas son soft delete.
- responsable_texto se usa cuando el responsable no existe en profiles (texto libre).
- transferencias.area_origen y area_destino son text (no FK) para preservar historial legible aunque se renombre un área.

6.3. Funciones y RPCs principales

Función	Tipo	Propósito
handle_new_user	trigger	Crea profiles automáticamente al registrar usuario.
handle_updated_at	trigger	Mantiene updated_at en bienes y profiles.
generar_codigo_bien	helper	Calcula correlativo por prefijo y año.

Función	Tipo	Propósito
get_my_rol / get_my_sede	helper	Evita recursividad en políticas RLS.
current_user_rol	helper	Permite al cliente consultar su propio rol.
require_rol_escritura	guard	Excepción si el caller no es ADMIN/ESTANDAR activo.
require_rol_admin	guard	Excepción si el caller no es ADMIN activo.
crear_bien_con_auditoria	RPC	Inserta bien + log de auditoría.
actualizar_bien_con_auditoria	RPC	Modifica bien + log.
crear_transferencia	RPC	Cambio de ubicación atómico (lock + log).
actualizar_rol_usuario	RPC	Cambia rol; protege último admin activo.
set_usuario_activo	RPC	Activa/desactiva; protege último admin activo.
listar_usuarios_admin	RPC	Lista usuarios con email de auth.users.

6.4. Roles y permisos

Rol	Lectura	Escritura inv.	Transfer.	Bajas	Catálogos / Usuarios
ADMINISTRADOR	Todo	Sí	Sí	Sí	Sí
ESTANDAR	Todo	Sí	Sí	No	No
CONSULTOR	Todo	No	No	No	No

6.5. Seed inicial

supabase/seed.sql es idempotente y carga:

- 4 sedes (Bogotá, Medellín, Cali, Barranquilla).
- 6 áreas (GAF, Focos, Tecnología, Talento Humano, Comunicaciones, Dirección).
- 9 tipos de bien con prefijos: COMP, PORT, MON, IMP, MOB, VID, TEL, RED, OTRO.

No crea usuarios. Editar el seed antes del primer db reset si se requieren otros catálogos.

6.6. Almacenamiento (Storage)

Bucket bienes — público, máximo 5 MB, formatos image/jpeg | png | webp:

- SELECT abierto para mostrar imágenes con URL pública.

- INSERT/UPDATE/DELETE solo authenticated.
- Las imágenes se suben directamente desde el cliente (evita el límite de 4.5 MB de Server Actions en Vercel).
- Nombre de archivo: {crypto.randomUUID()}.ext en raíz del bucket.

7. Módulos funcionales

Ruta	Módulo	Acceso
/	Portada pública	público
/auth/*	Autenticación	público
/inicio	Panel con KPIs, timeline y gráfico por sede	todos
/bienes	Listado, alta, edición, detalle y filtros	lectura: todos · escritura: ADMIN/ESTANDAR
/sedes /areas /categorias	CRUD de catálogos	lectura: todos · escritura: ADMIN
/transferencias	Cambios de ubicación entre sedes/áreas/responsables	lectura: todos · escritura: ADMIN/ESTANDAR
/bajas	Salida de inventario con confirmación doble	ADMIN
/historial	Timeline cronológico por bien	todos
/reportes	Inventario por persona	todos
/usuarios	Gestión de roles y activación	ADMIN
/api/export/*	Descarga de archivos .xlsx	según ruta

7.1. Reglas de negocio destacadas

- Transferencias: solo bienes ACTIVO. No se permite transferir a la misma combinación sede + área + responsable.
- Bajas: solo bienes ACTIVO, solo ADMINISTRADOR. Transacción atómica que cambia el estado, registra en bajas e inserta en movimiento_bienes.
- Auditoría: cada operación (REGISTRO, MODIFICACION, TRANSFERENCIA, BAJA) deja un registro escrito por el RPC correspondiente.
- Último admin activo: las RPCs actualizar_rol_usuario y set_usuario_activo validan que la operación no deje el sistema sin administradores.

8. Autenticación y autorización

8.1. Flujo de auth

7. Login con email/password vía `supabase.auth.signInWithPassword()`.
8. Supabase emite un JWT y lo guarda en cookies HTTP-only (gestionado por `@supabase/ssr`).
9. `proxy.ts` (middleware de Next.js) refresca la sesión en cada request a rutas no públicas.
10. El rol se obtiene de `profiles.rol` cuando se necesita (en el layout del dashboard, en `getAuthContext`, o en RPCs vía `get_my_rol()`).

8.2. Guard de página (server component)

```
import { requireRol, ADMIN_ONLY } from "@/lib/auth/require-rol";

export default async function BajasPage() {
  const ctx = await requireRol(ADMIN_ONLY);
  // si no es ADMIN, requireRol ya hizo redirect("/inicio")
}
```

8.3. UI condicional por rol

```
const ctx = await getAuthContext();
const canWrite =
  ctx.rol === ROLES.ADMINISTRADOR ||
  ctx.rol === ROLES.ESTANDAR;
```

9. API y endpoints

El proyecto no expone una API REST pública. Las mutaciones internas se hacen vía Server Actions. Los únicos endpoints HTTP propios son Route Handlers de descarga:

Método	Endpoint	Devuelve
GET	/api/export/bienes	.xlsx con todos los bienes (filtros vía query string).
GET	/api/export/inventario-persona	.xlsx con inventario por responsable.
GET	/api/export/historial?id_bien=N	.xlsx con la trazabilidad de un bien.

Todos exigen sesión activa; sin sesión devuelven redirect a `/auth/login`. El consumo del cliente con la base de datos pasa siempre por `@supabase/ssr` y `@supabase/supabase-js`.

10. Despliegue

10.1. Frontend en Vercel

11. Conectar el repo a un proyecto de Vercel.
12. Configurar variables de entorno (Project Settings → Environment Variables):
NEXT_PUBLIC_SUPABASE_URL y NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY.
13. Build command: next build (default).
14. Cada push a master dispara un deploy automático.

10.2. Backend en Supabase Cloud

15. Crear proyecto en supabase.com.
16. supabase link --project-ref <ref>.
17. supabase db push para aplicar la migración baseline.
18. Cargar el seed con psql "\$DATABASE_URL" -f supabase/seed.sql.
19. El bucket bienes queda configurado por la migración.

10.3. Migraciones futuras

```
supabase migration new <nombre_descriptivo>
# editar el SQL generado
supabase db reset          # aplicar en local
supabase db push           # aplicar en remoto
```

La carpeta supabase/migrations/_archive/ no se vuelve a ejecutar — es solo memoria histórica.

11. Pruebas

```
npm run lint      # ESLint
npm run build     # build + type-check
npm run test:unit # Vitest (validaciones, helpers)
npm run test:e2e  # Playwright (flujos base)
npm run check     # lint + build + test:unit
```

Los E2E autenticados leen E2E_USER_EMAIL y E2E_USER_PASSWORD de .env.local. Sin esas variables, Playwright solo valida que las rutas protegidas y las descargas redirijan al login.

12. Convenciones de código

Elemento	Convención	Ejemplo
Archivos / carpetas	kebab-case	bien-form.tsx
Componentes React	PascalCase	BienForm
Hooks	camelCase con use	useBienes
Variables / funciones	camelCase	valorTotal
Tipos / interfaces	PascalCase	CreateBienInput
Constantes	UPPER_SNAKE_CASE	ESTADOS_BIEN
Tablas BD	snake_case plural	bienes
Columnas BD	snake_case	valor_unitario
Rutas URL	kebab-case	/bienes/nuevo

12.1. Reglas duras

- TypeScript strict. Prohibido any (preferir unknown).
- Server Components por defecto. "use client" solo cuando se requiera interactividad.
- Una mutación que toca más de una tabla pasa por un RPC PL/pgSQL (security invoker).
- Nunca eliminar registros físicamente: soft delete cambiando estado.
- No hardcodear strings de roles: usar ROLES.* de lib/constants.ts.
- Inglés para código, español para UI.

13. Mantenimiento

13.1. Tareas periódicas

- Backups: Supabase Cloud genera backups diarios automáticos. Descargar respaldo manual antes de migraciones grandes.
- Actualización de dependencias: revisar npm outdated mensualmente. Mayor riesgo: next, @supabase/*, react.
- Auditoría de roles: revisar trimestralmente la lista de ADMINISTRADOR y desactivar usuarios que ya no estén en la organización.
- Consumo de Storage: monitorear el bucket bienes desde el dashboard de Supabase.
- Logs: revisar logs de Supabase y Vercel ante reportes de error.

13.2. Puntos de extensión

Necesidad	Dónde tocar
Nuevo módulo	app/(dashboard)/<nuevo>/ + entry en NAV_GROUPS (lib/constants.ts).
Nueva validación	lib/validations/<entidad>.ts (Zod).
Nueva regla de BD	Nueva migración en supabase/migrations/ + RPC si modifica datos.
Nuevo reporte Excel	app/api/export/<reporte>/route.ts + helper en lib/export/excel.ts.
Nuevo rol	CHECK constraint en profiles.rol, RPCs require_rol_*, ROLES en constants.ts y políticas RLS.

13.3. Recuperación ante desastres

20. Recrear proyecto Supabase y ejecutar la migración baseline.
21. Restaurar el respaldo más reciente desde el dashboard de Supabase.
22. Verificar el bucket bienes: las imágenes están vinculadas por URL pública; si se restaura el dump, las URLs siguen siendo válidas.
23. Revisar variables de entorno en Vercel.
24. Volver a desplegar el frontend.

14. Referencias internas

- README.md — guía rápida de arranque.
- AGENTS.md — guía exhaustiva de arquitectura y convenciones para colaboradores.
- documentation/CODE_GUIDE.md — recorrido del código para nuevos colaboradores.
- supabase/README.md — setup detallado de la BD y modelo de roles.
- documentation/CHANGELOG.md — historial de cambios.

Documento mantenido en documentation/MANUAL_TECNICO.md. Actualizar al introducir cambios estructurales.